



Indian Institute of Science, Bangalore

SL 203 - Introduction to Algorithms and Software Programming

CardioScheduler

Multi-Strategy Scheduling and Benchmarking System
for Cardiac-Ward Clinical Workflows

By: Subash C,M.Tech(Research) Student

Course Instructors: Nagarajan Natarajan, PhD — Tarun Rambha, PhD

April 2026

Problem Motivation

Today, in most Indian wards.

- Pen-and-paper task lists.
- “First on the page gets done first.”
- No escalation when a CRIT patient’s followup.
- No record of *why* a schedule was chosen.

The cost.

- Healthcare-worker burnout.
- Delayed urgent care.
- No feasibility verdict before the shift starts.

--- Ward Roster (paper) ---

[]	T_ECG	P004	Nurse R.
[]	T_TROP	P017	Intern S.
[]	T_ECHO	P001	Nurse R.
[]	T_CXR	P012	Intern T.
[]	T_STRESS	P007	PGY1 K.
[]	T_BMP	P022	Nurse R.
	...	...	...

- first row gets done first
- no priority field
- no shift / capacity check
- re-triage = scratch & rewrite

The hospital floor is a queueing system whose tasks aren't interchangeable.

- Patients differ in *acuity* (CRIT/HIGH/MED/LOW) and *disease*; both shape the workup.
- Tests have hard *precedence* (Troponin → ECHO → Stress).
- Workers are *role-typed* (PGY1, INTERN, NURSE) and shift-bounded.
- Shift exceeded $\Rightarrow$ overtime; overtime exceeded $\Rightarrow$ burnout.

What an algorithm-driven scheduler must do. Respect roles — respect precedence — minimise urgent-patient wait — honour finite shifts — report feasibility loudly.

Existing Gaps in Current Practice

Static heuristics.

- One scalar priority $\Rightarrow$ entire Pareto regions unreachable.
- No notion of role-aware matching.
- No way to compare two algorithm variants on the same input.

No defensible accountability.

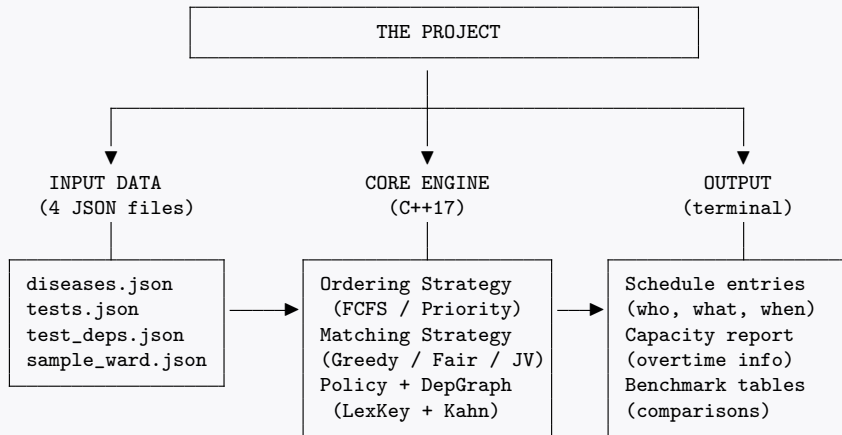
- No feasibility verdict — shifts silently overrun.
- No per-acuity timing report.
- No record of *which trade-offs* were chosen and *why*.

claim: a scheduling engine that can be *benchmarked* on multiple axes is more valuable than one that ships “the optimal algorithm”.

Project Goals

- 1 **Correct, deterministic discrete-event scheduler** with two pluggable policy axes (ordering, matching).
- 2 **Lexicographic priority abstraction** (LexKey) — a hand-curated 7-policy catalogue subsuming the legacy scalar ordering.
- 3 **Multi-objective evaluation**: Pareto frontier on (urgent-care, makespan, max-OT) + NSGA-II crowding.
- 4 **Six-mode benchmark suite** that isolates one algorithmic axis at a time.

System Overview — Input → Engine → Output



100 patients $\times$ $\sim$ 20 tests/disease = 2,017 task instances at runtime.

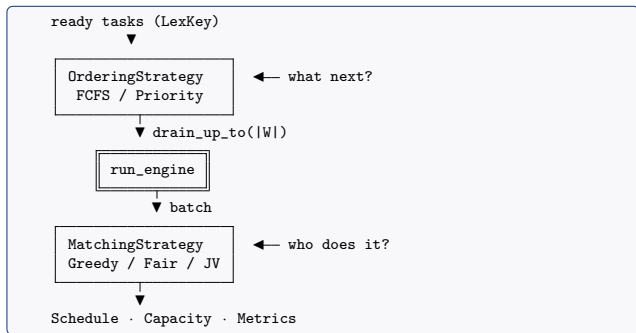
Architecture — The Engine, One Tick at a Time

Six things every tick.

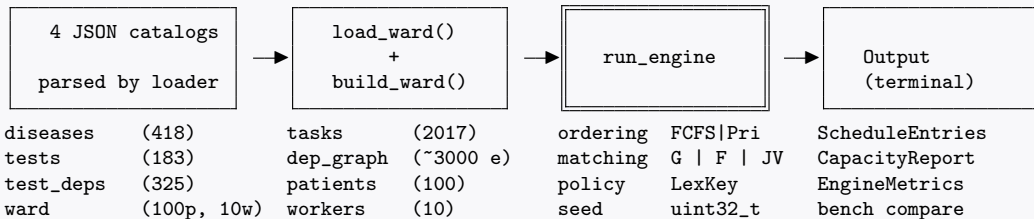
- 1 Look at unfinished tasks.
- 2 Check rules: which are ready?
- 3 Sort the ready set (ordering).
- 4 Find a free, role-matched worker.
- 5 Assign; move time forward.
- 6 Unblock dependents; loop.

Two pluggable axes:

ordering (what next) — **matching** (who does it).



Pipeline Walkthrough — Data Flow



Key Modules

Module	Responsibility
<code>data/loader.cpp</code>	JSON parsing, ward assembly
<code>data_structures/graph</code>	Append-only <code>DirectedGraph</code>
<code>scheduler/topological</code>	Kahn's cycle check
<code>scheduler/allocator</code>	<code>run_engine</code> , capacity report
<code>scheduler/strategies</code>	Ordering + matching strategies
<code>scheduler/hungarian</code>	JV min-cost assignment
<code>scheduler/policies</code>	LexKey catalogue + Pareto tools
<code>bench.cpp</code>	Legacy / 2×2 / Pareto / Fair benches
<code>bench_adversarial.cpp</code>	Synthetic pathological scenarios
<code>bench_urgency.cpp</code>	Per-acuity-tier timing across 5 matchers
<code>main.cpp</code>	CLI & dispatch

Data Processing

Inputs.

File	Shape	Count
diseases.json	disease → test list, base-priority	418
tests.json	test template (role, duration, priority)	183
test_dependencies.json	global must-precede edges	325
sample_ward.json	100 patients + 10 workers	100/10

Build-time denormalisation.

- Each patient gets their own copy of every test in their disease's protocol.
- Task IDs are suffixed by patient (e.g. T_ECG_P001 = "ECG for P001").
- Precedence edges are duplicated inside each patient's sub-graph; *patients never block each other.*

Algorithms (1/2) — Ordering and Lex Policy

Ordering: FCFS **vs.** Priority.

- FCFS = sorted-by-id deque (deterministic FIFO).
- Priority = min-heap on LexKey.

LexKey = lexicographic priority key (smaller = more urgent).

Policy	Key shape
acuity_first	(acuity, task.priority, hash)
priority_first	(task.priority, acuity, hash)
acuity_short_first	(acuity, +duration, ..., hash)
acuity_long_first	(acuity, -duration, ..., hash)
acuity_old_first	(acuity, -age, ..., hash)
acuity_young_first	(acuity, +age, ..., hash)
duration_first	(+duration, ..., hash)

Why lex over scalar: a scalar weighted-sum cannot pick between two schedules that agree on the dominant axis. Lex keys can.

Algorithms (2/2) — Matching

Three matchers, one interface.

Matcher	Rule (in plain English)
Greedy	Pick the role-matched worker who's free earliest.
FairGreedy(α)	Blend “earliest free” with “most shift left” (pure water-fill).
Hungarian	Globally optimal 1-to-1 pairing per tick.

Per-tick behaviour matters. Greedy can stack many tasks on one worker in a single tick; Hungarian commits at most one task per worker per tick, so overflow tasks defer to the next tick. This single difference drives the empirical results we'll see next.

Build. Using CMake. **CLI — one binary, six bench modes.**

Flag	Bench mode
(none)	Legacy: FCFS vs. Priority
--bench-2x2	ordering $\times$ matching matrix
--bench-pareto	7 lex policies $\times$ 5 seeds
--bench-fair	throughput vs. intra-role stdev
--bench-adversarial	4 hand-built pathological scenarios
--bench-urgency	per-acuity timing, 5 matchers

Experimental Setup — Production Ward

Parameter	Value
Patients	100 (P001--P100)
Acuity mix	CRIT 23 — HIGH 28 — MED 49 — LOW 0
Workers	10 (2 NURSE — 5 INTERN — 3 PGY1)
Worker shifts	8 × 480 min, 2 × 600 min
Catalogues	418 diseases / 183 tests / 325 dep edges
Per-patient task instances	2,017
Total roster capacity	5,040 min
Implied workload	~25,000+ min ($\approx 5\times$ over)
Total OT (every strat)	20,210 min

Run discipline. Every cell deep-copies tasks/workers/patients; constructs fresh strategy instances; measures wall-clock with `steady_clock`.

Results 1 — Legacy Bench (FCFS vs. Priority)

CRIT+HIGH metric (min)	FCFS	Priority	Δ
First task start	134.0	155.1	+21
Last task finish	8,481.4	3,103.9	−5,377
Total overtime	20,210	20,210	0
Capacity breaches	1,827	1,820	−7

Read. Priority delays the *first* urgent task by ~ 21 min but finishes urgent patients $\sim 5,378$ min sooner on average.

Caveat. Same total work; same total OT — ordering redistributes who waits.

Results 2 — 2×2 Ordering × Matching

Cell	Combination	Makespan	OT	Breaches	Runtime	Re-q.
A	FCFS × Greedy	9,260	20,210	1,827	12.0 ms	—
B	FCFS × Hungarian	9,350	20,210	1,830	65.5 ms	7,243
C	Priority × Greedy	9,260	20,210	1,820	15.9 ms	—
D	Priority × Hungarian	9,470	20,210	1,819	84.2 ms	6,939

Pairwise deltas.

- A→C (matching held Greedy): $\Delta\text{ms} = 0 \Rightarrow$ ordering-only win is 0 min on makespan.
- A→B (ordering held FCFS): $\Delta\text{ms} = +90 \Rightarrow$ Hungarian *costs* makespan.
- C→D (Hungarian over Priority): $\Delta\text{ms} = +210$ and $\sim 3.6\times$ wall-clock — not worth the complexity here.

Results 3 — Pareto Sweep over Lex Policies

Policy	Best CRIT+HIGH last (min)	Δ vs. default
acuity_young_first	2,695.9	−408 (−13%)
acuity_old_first	2,714.4	−390
acuity_first (default)	3,103.9	baseline
priority_first	3,395.0	+291
acuity_short_first	3,562.0	+458
acuity_long_first	3,604.4	+500
duration_first	7,910.3	+4,806 (worst)

Headline. A $2.9\times$ spread on the urgent-care metric for the *same input*. `acuity_first` and `acuity_young_first` agree on dim-1 — only the age tie-break in dim-2 separates them. *No scalar weighted-sum could pick between the two.*

Results 4 — Fairness Sweep (Throughput vs. Intra-role Load)

Priority ordering held; matcher varies.

Matcher	Throughput	Load stdev	Intra-role	Max-OT	Runtime
Greedy	0.2178	3,467.9	7.1	8,780	15.9 ms
Hungarian	0.2130	3,469.2	73.5	8,990	84.5 ms
Fair($\alpha=0.75$)	0.2178	3,467.9	12.6	8,780	15.7 ms
Fair($\alpha=0.50$)	0.2178	3,468.0	21.8	8,780	15.7 ms
Fair($\alpha=0.25$)	0.2178	3,468.1	32.0	8,780	15.7 ms
Fair($\alpha=0.00$)	0.2178	3,468.3	42.0	8,780	15.7 ms

Read. Greedy is the sole strict-Pareto winner on (throughput $\uparrow$, intra-role stdev $\downarrow$). Every worker starts fresh at $T=0$, so there is nothing for water-filling to redistribute. The mechanism is correctly wired but inert on this dataset.

Results 5 — Urgency Bench (per-Acuity Timing)

Matcher	CRIT mean	p_{90}	Worst CRIT
Greedy	1,795	2,260	2,415
Hungarian	3,318 (+85%)	9,280 (+311%)	9,470 (4×)
Fair($\alpha=0.75$)	1,795	2,260	2,415
Fair($\alpha=0.50$)	1,795	2,260	2,415
Fair($\alpha=0.25$)	1,795	2,260	2,415
Fair($\alpha=0.00$)	1,795	2,260	2,415

Hungarian is *actively bad* for critical patients here. 1-to-1-per-tick deferral on identical-key CRIT bursts. Greedy stacks them in one tick. Clinical correctness, not just runtime.

Key Insights

- 1 **Ordering moves more than matching** on a roster past capacity. Tie-breaks dominate.
- 2 **Globally-optimal-per-tick is the wrong invariant** when priority keys collide. Greedy stacks; Hungarian defers.
- 3 **Fair-mode water-filling needs heterogeneous initial load to engage.** Mechanism correct; dataset doesn't exercise it.
- 4 **No scheduling change creates capacity.** 20,210 min of overtime are invariant.

Recommended production config. Priority + Greedy + acuity_young_first Lex-Policy.

Recommended staffing change. Add nurses. The roster is at $\sim 5\times$ overload.

Failure Cases & Adversarial Behaviour

Four hand-built pathological wards.

Scenario	Breaks
all-critical	Priority degenerates to tie-hash (every key=11)
long-chain	Parallelism = 1; matchers can't differentiate
single-role-bottleneck	All work on one worker; Hungarian wastes ticks
tight-shift	Zero slack; perfect-split with no margin

Schedule quality is identical across all five strategies on every scenario. Only runtime differs. Hungarian costs $2\text{--}4\times$ Greedy with no schedule improvement.

Limitations

Scope cuts (intentional).

- Static acuity (no re-triage).
- Deterministic durations.
- Single-snapshot model.
- Per-disease (not per-task) priority.
- Hardcoded role / acuity enums.

Engineering / empirical.

- No automated test suite (CTest).
- No reusable library target.
- Findings conditional on one production ward.
- Acuity mix is critical-skewed (no LOW patients).
- Adversarial bench bypasses
`--overtime-threshold`.

Future Improvements

- 1 **Stack-aware Hungarian.** Restore the optimality guarantee while eliminating CRIT deferrals.
- 2 **Acuity escalation path.** Re-triage event handler; re-key the priority heap mid-run.
- 3 **Probabilistic durations.** Monte-Carlo schedules; report mean / p_{95} rather than a single deterministic schedule.
- 4 **Reusable library + CTest** for the JV solver, topological sort, `compute_objectives`.
- 5 **Multi-instance benchmarking.** 5–10 ward snapshots spanning roster mixes.
- 6 **Clinical validation** with a practising cardiologist.

Conclusion

What I implemented.

- A discrete-event scheduler with two pluggable policy axes (ordering, matching).
- Three matchers (Greedy, FairGreedy, Hungarian-JV) and a seven-policy lexicographic priority catalogue.
- A multi-objective Pareto frontier with NSGA-II crowding selection.
- A six-mode benchmark suite over a 100-patient, 10-worker ward.

What the experiments showed.

- On this dataset, the ordering axis moves the urgent-care metric more than the matching axis does.
- Lex tie-breaks reach Pareto points that a scalar weighted-sum priority cannot.
- Hungarian's per-tick optimality regressed CRIT timing on this dataset — a 1-to-1-per-tick contract issue, not a solver bug.
- Total overtime is invariant across every strategy — a workforce question, not an algorithm question.

Thank you.

Open for discussion.
